

Algoritmi e Strutture Dati – Parte A – 08/02/2021**Esercizio A1 – Punti ≥ 8**

Trovare i limiti superiore e inferiore più stretti possibili per la seguente equazione di ricorrenza:

$$T(n) = \begin{cases} 1 & n < 8 \\ 3T(\lfloor n/2 \rfloor) + 3T(\lfloor n/4 \rfloor) + n^2 \log n & n \geq 8 \end{cases}$$

Esercizio A2 - Closest - Punti ≥ 10

Scrivere un algoritmo

int closest(**int**[] *A*, **int** *n*, **int** *x*)

che prenda in input un vettore *A* contenente *n* interi *ordinati* e restituisca il valore di *A* (o uno dei valori, se più di uno) la cui distanza da *x* (in valore assoluto) sia la più piccola possibile.

Spiegare il funzionamento e discutere la complessità computazionale dell'algoritmo proposto. *Soluzioni in tempo lineare o superiore non verranno considerate.*

Ad esempio, con $A = [-3, -1, -1, 2, 4, 7]$:

- se $x = 0$, l'algoritmo deve restituire -1 ;
- se $x = 1$ oppure $x = 2$, deve restituire 2
- se $x = 3$ deve restituire 2 oppure 4 .

Esercizio A3 – Conversione numeri – Punti ≥ 12

Scrivere un algoritmo

int minOps(**int** *n*, **int** *m*)

che prenda in input due numeri positivi *n* e *m*, con $n < m$, e restituisca il numero *minimo* di operazioni necessarie per trasformare il numero *n* nel numero *m*, assumendo che siano possibili due operazioni:

- Potete moltiplicare il numero per 2
- Potete sottrarre 1 al numero

Discutere correttezza e complessità computazionale dell'algoritmo proposto.

Esempio: è possibile trasformare 3 in 7 con la seguente sequenza: $3 \rightarrow 2 \rightarrow 4 \rightarrow 8 \rightarrow 7$, che richiede 4 operazioni ed è minima.

Esistono altre sequenze, come ad esempio $3 \rightarrow 6 \rightarrow 12 \rightarrow 11 \rightarrow 10 \rightarrow 9 \rightarrow 8 \rightarrow 7$, che richiede 7 operazioni e non è minima.

Algoritmi e Strutture Dati – Parte B - 08/02/2021**Esercizio B1 - Somma di primi - Punti ≥ 9**

Scrivere un algoritmo

`primes(int[] P, p, k, n)`

che prenda in input tre interi p , k , n e un vettore P di dimensione p contenente tutti i numeri primi inferiori o uguali a n in ordine, e che stampi tutti i modi per ottenere il numero n come somma esattamente di k numeri primi distinti.

Discutere correttezza e complessità dell'algoritmo proposto.

Ad esempio, con $n = 24$ e $k = 3$ bisognerà stampare $2 + 3 + 19$ e $2 + 5 + 17$.

Esercizio B2 – Zaino leggero – Punti ≥ 9

Scrivere un algoritmo

`int smallSum(int[] W, int n, int C)`

che prenda in input un vettore di pesi W di dimensione N e una capacità C e restituisca la dimensione del più piccolo sottoinsieme $S \subseteq \{1, \dots, n\}$ tale per cui $\sum_{i \in S} W[i] = C$.

Discutere correttezza e complessità dell'algoritmo proposto.

Se non è possibile ottenere C l'algoritmo deve restituire ∞ ; oppure potete assumere che sia sempre possibile ottenere C in almeno un modo. Scegliete l'approccio che preferite.

Ad esempio, se $W = [5, 1, 7, 2, 3, 9, 8]$ e $C = 17$, è possibile ottenere C come $2 + 3 + 5 + 7$, $1 + 3 + 5 + 8$, $2 + 7 + 8$, $1 + 2 + 5 + 9$, $3 + 5 + 9$, $1 + 7 + 9$, $8 + 9$. L'ultima somma è ottenuta con il minor numero di termini e si dovrà restituire 2.

Esercizio B3 – Trova l'ordine – Punti ≥ 12

Si scriva un algoritmo

`int[] findOrder(int[] A, ITEM[] D, int n)`

che prenda in input un vettore A contenente n interi distinti, non necessariamente ordinati e un vettore A contenente $n - 1$ simboli di disequazione ' $<$ ' oppure ' $>$ ', e restituisca una permutazione A' di A tale per cui:

- se $D[i] = '<'$ allora $A'[i] < A'[i + 1]$;
- se $D[i] = '>'$ allora $A'[i] > A'[i + 1]$.

Suggerimento: una permutazione del genere esiste sempre. Tentare di dimostrare per induzione che tale permutazione esiste sempre può essere un modo per identificare un algoritmo efficiente, altrimenti date semplicemente per scontato che tale permutazione esista.

Discutere correttezza e complessità dell'algoritmo proposto.

Ad esempio, se $A = [3, 23, 5, 12, 32, 7]$ e $D = [<, >, <, >, >]$, una possibile permutazione è $[3, 32, 5, 23, 12, 7]$, in quanto $3 < 32 > 5 < 23 > 12 > 7$.